

Задание 23 ЕГЭ Итоговое повторение

Информатика



Задачи

Задача 1

Задача

Исполнитель Счастье умеет только прибавлять 1 и умножать на 2.
Сколько существует различных вариантов получения числа 9 из числа 3?
Решите задачу без программирования.

Задача 1: Решение

Отобразим варианты решения с помощью дерева команд ручным способом:

$$F(3) = 1 \text{ (старт)}$$

$$F(4) = F(3) = 1$$

$$F(5) = F(4) = 1$$

$$F(6) = F(5) + F(3) = 1 + 1 = 2$$

$$F(7) = F(6) = 2$$

$$F(8) = F(7) + F(4) = 2 + 1 = 3$$

$$F(9) = F(8) = 3$$

Ответ

3

Задача 2

Задача

В теории графов вершина, через которую обязательно должны проходить все пути, называется точкой сочленения.

А как называется дуга (ребро), через которую должны проходить все пути?

Задача 2: Решение

В теории графов дуга или ребро, удаление которого увеличивает число компонент связности (и соответственно, это единственное ребро, связывающее две части, через которое обязаны проходить пути), называется **мостом**.

Ответ

МОСТ

Задача 3

Задача

Сколько существует программ, для которых при исходном числе 7 результатом является 88, при этом траектория вычислений содержит хотя бы одно из чисел 23 или 45?

Выберите верные варианты решения:

1. Вызвать рекурсивную функцию с обязательным значением 23, затем с обязательным значением 45. Результаты сложить.
2. Вызвать рекурсивную функцию с обязательным 23, затем с обязательным 45 и исключаемым 23. Результаты сложить.
3. Вызвать рекурсивную функцию без условий. При прохождении 23 или 45 "поднимать флаг". Подсчитывать траектории только с поднятым флагом.
4. Вызвать рекурсивную функцию с обязательным 45, затем с обязательным 23 и исключаемым 45. Результаты перемножить.
5. Подсчитать все варианты траекторий от 7 до 23, от 23 до 45, от 45 до 88. Результаты перемножить.

Задача 3: Решение

Чтобы учесть маршруты, проходящие *хотя бы через одно* из чисел 23 или 45, мы можем:

- Использовать формулу включений-исключений (разбить на непересекающиеся множества): маршруты проходящие через 23 + маршруты, проходящие через 45 *и не проходящие* через 23. (Вариант 2).
- Добавить дополнительный параметр в функцию (флаг), который становится True, если мы хоть раз оказались в точке 23 или 45, и засчитывать только такие ветви. (Вариант 3).

Ответ

Варианты 2 и 3

Задача 4

Задача

Сколько существует программ, для которых при исходном числе 106 результатом является 6, при этом траектория вычислений содержит ровно одно из чисел 46 или 61 (но не оба одновременно)?

Выберите правильный вариант решения:

1. Подсчитать все варианты траекторий от 106 до 61, от 46 до 6. Результаты сложить.
2. Вызвать рекурсивную функцию с обязательным 46, затем с обязательным 61 и исключаемым 46. Результаты сложить.
3. Подсчитать все варианты траекторий от 106 до 61, от 61 до 46, от 46 до 6. Результаты сложить.
4. Вызвать рекурсивную функцию с обязательным 46 и исключаемым 61, затем с обязательным 61 и исключаемым 46. Результаты сложить.

Примечание: в задаче используются числа 46 и 61 в вариантах ответа.

Задача 4: Решение

Ограничение "строго одно из двух" означает, что нам подходят пути, которые проходят через первое число и избегают второе, ПЛЮС пути, которые избегают первое число и проходят через второе. Этому в точности соответствует алгоритм в Варианте 4.

Ответ

Вариант 4

Задача 5

Задача

Сколько существует программ (команды: +1, +2, *2), для которых при исходном числе 3 результатом является число 20, при этом траектория вычислений содержит число 7 и не содержит 10? Восстановите программу, выбрав правильные элементы: `s == f`, `&`, `s > f`, `-`, `or`, `+`, `*`, `s < f`, `and`, `s <= f`

```
def ways(s, f):  
    if [1] : return 1  
    if [2] [3] s == 10: return 0  
    return ways(s + 1, f) [4] ways(s + 2, f) [5] ways(s * 2, f)  
  
print(ways(3, 7) [6] ways(7, 20))
```

Задача 5: Решение

Для стандартной функции прямого пересчёта (команды увеличивают число) мы:

- Возвращаем 1, когда начальное значение сравнялось с конечным: $s == f$.
- Возвращаем 0, если "перепрыгнули" цель $s > f$ ИЛИ (or) оказались в исключаемом числе ($s == 10$).
- Складываем (+) результаты всех рекурсивных вызовов команд от ветвей.
- Перемножаем (*) этапы обязательного прохождения (от старта к точке, и от точки к финишу).

Ответ

1) $s == f$, 2) $s > f$, 3) or, 4) +, 5) +, 6) *

Задача 6

Задача

У исполнителя Счастье есть две команды: +2, +5. Определите число, для получения которого из числа 5 существует 34 программы. Какое логическое выражение должно быть в программе на месте многоточия?

```
def ways(s, f):  
    # функция, возвращающая количество программ  
    for i in range(100):  
        if ... :  
            print(i)
```

Варианты: 1) $\text{ways}(5, 34) == i$, 2) $\text{ways}(5, i) == 34$, 3) $\text{ways}(i, 34) == 5$, 4) $\text{ways}(5, i) * \text{ways}(i, 34) == 1$

Задача 6: Решение

Чтобы найти числа i , количество путей для получения которых из стартовой точки 5 равняется 34, мы перебираем конечное число i в цикле и вызываем функцию `ways(стартовая_точка, проверяемая_точка)`.

Если результат функции равен 34, то число i — искомое. Код: `ways(5, i) == 34`.

Ответ

Вариант 2: `ways(5, i) == 34`

Задача 7

Задача

Решите задачу, построив дерево. У исполнителя Счастье имеются две команды, которым присвоены номера: 1) Прибавить 2. 2) Умножить на 3.

Сколько различных чисел можно получить из числа 1 с помощью четырёх команд?

Задача 7: Решение

Для решения сбора всех возможных элементов дерева и исключения дубликатов удобно использовать множества (set) в Python:

Python

```
s = {1}
for _ in range(4):
    s = {x + 2 for x in s} | {x * 3 for x in s}
print(len(s))
```

Данный код выполняет команды ровно 4 раза над текущим уникальным набором чисел.

Ответ

8

Задача 8

Задача

Сколько существует программ, состоящих из 7 команд (+1, +2, +3), для которых при исходном числе 3 результатом является число 22? Восстановите программу:

```
def ways(s, f, n):  
    if s == f [1] n [2] 7: return 1  
    if s > f [3] n [4] 7: return 0  
    return ways(s + 1, f, n + 1) + ways(s + 2, f, n + 1) + ways(s + 3, f, n + 1)  
print(ways(3, 22, 0))
```

Задача 8: Решение

Для точного ограничения на количество команд мы используем дополнительный аргумент n .

- Окончание успешное: мы достигли нужной суммы И (and) совершили ровно 7 шагов ($== 7$).
- Окончание провальное: мы "перепрыгнули" искомое число ИЛИ (or) совершили уже 7 шагов ($== 7$) без достижения точки финиша, либо сделали больше шагов.

Ответ

1) and, 2) $==$, 3) or, 4) $- / ==$

Задача 9

Задача

Исполнитель преобразует число (команды: $+1$, $+3$, $*2$). Сколько существует программ, которые преобразуют число 3 в 30 и при этом **не содержат двух команд « $+1$ » подряд**? Выберите правильный алгоритм решения задачи:

1. Эта задача решается только вручную, построением дерева.
2. Вызвать рекурсивную функцию. Если последняя команда 3, то на следующем уровне вызывается только одно поддерево (« $*2$ »), иначе — все 3 поддерева.
3. Вызвать рекурсивную функцию от 3 до 30. В качестве параметра передавать номер последней команды. Если это 1, то на следующем уровне вызываются только два поддерева (« $+3$ » и « $*2$ »), иначе — все 3 поддерева.

Задача 9: Решение

Ограничение "не содержат двух команд подряд" обрабатывается путём передачи в рекурсию дополнительного параметра — номера последней выполненной команды. Если предыдущая команда уже была под номером 1 («+1»), то в текущем вызове мы не имеем права вызывать функцию с командой «+1» заново. Зато мы имеем право использовать любые другие: «+3» и «*2».

Ответ

Вариант 3 (Если это 1, то на следующем уровне вызываются только два поддерева («+3» и «*2»))

Задача 10

Задача

У исполнителя Счастье имеются три команды: -3 , -6 , и найти целую часть от деления на 2. Сколько существует программ, для которых при исходном числе 86 результатом является 12, при этом траектория вычислений не содержит 36 и содержит 53?

Задача 10: Решение

Программа работает с убывающими числами, поэтому условие отрицательной базы окончания: $s < f$.

Python

```
def ways(s, f):  
    if s == f: return 1  
    if s < f or s == 36: return 0  
    return ways(s - 3, f) + ways(s - 6, f) + ways(s // 2, f)  
  
print(ways(86, 53) * ways(53, 12))
```

Ответ

144